

SD TCL - The edit Screen Editor

Microsoft Edit, a menu bar and a dozen keys — the one to reach for when you just want to change a record.

```
edit {dict} file record
```

`edit` opens a record in **Microsoft Edit**, a small full-screen editor that ships with Windows. It has a menu bar, the shortcuts everyone already knows, and nothing else to learn. **That is the point of it:** if you want to fix a line in a record and get on with your day, this is the one.

It has no syntax highlighting and no command language. For SD BASIC source — or for anything where you want highlighting, split windows, or a command bar — use `micro` instead. For a session with no terminal, or one you are driving from a script, use `ed`.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The keys below are Microsoft Edit's own, for the version SD checks for — **1.2.1**, which is what ships in current Windows. SD installs that editor and calls it; it does not implement it, so where a binding differs the editor is right.

Both editors are installed with SD

You do not install anything. SD's installer checks for both editors and installs whichever is missing, machine-wide, so every account SD creates can reach them. Microsoft Edit is usually already there — it is part of current Windows, at

`C:\Windows\System32\edit.exe`.

If that could not happen — an offline machine, or one whose policy blocks the package manager — the verb says so and names the command that installs it, rather than opening nothing and reporting the record unchanged.

The keys

`Ctrl-S` saves and `Ctrl-Q` quits. Those two are most of what anyone needs.

`Ctrl-S`

save

Ctrl-Q	exit
Ctrl-W	close the file
Ctrl-O · Ctrl-N	open a file · new file
Ctrl-Z · Ctrl-Y	undo · redo
Ctrl-X · Ctrl-C · Ctrl-V	cut · copy · paste
Ctrl-A	select all
Ctrl-F · Ctrl-R	find · replace
Ctrl-G	go to line:column
Ctrl-P	go to file
Alt-Z	toggle word wrap

The menu bar is the help. There is no help screen — the Help menu holds only *About* — and there does not need to be one: **F10** or **Alt** and the menu's letter opens a menu, and every command is listed there **with its shortcut printed beside it**.

Alt-F	File
Alt-E	Edit
Alt-V	View
Alt-H	Help

Save As is on the File menu and is the one common command with no shortcut of its own: **Alt-F** then **A**.

What SD does around the editor

The two screen editors are **one SD program with two names**, so everything in this section is equally true of micro.

the working copy the record is copied into `$hold` as *record*.`editing`, and the editor is run on that. It is removed on every exit, including the ones that fail

saving	"Save? <Y>es, <N>o", then for a <code>bp</code> record "Compile?" and "Catalogue?"
a <code>dict</code> record	is always saved and re-compiled with <code>cd</code>
finishing	"<E>xit or <R>e-edit", so a compile error can be fixed without starting again

A compiled dictionary record is truncated to its first 15 fields while you edit it, which is what you want: the fields after them are the compiled form, and `cd` rebuilds them when you save.

Marks, and how to type one

A **field** mark is a line break, so a text editor handles fields on its own. The other three marks are single control characters an editor would either draw as a stray glyph or drop, so each has a token you type instead.

Every token is `~` and one more character, and `~` is the only escape character.

<code>~~</code>	a value mark
<code>~`</code>	a subvalue mark
<code>~!</code>	a text mark
<code>~-</code>	a literal <code>~</code> , where one would otherwise be misread
<code>~,</code>	a literal <code>,</code> , where one would otherwise read as a separator

`SMITH~~JONES~~BROWN` is a three-value field; `RED~`BLUE~~GREEN` is two values, the first with two subvalues.

MARKS IN A ROW ARE SEPARATED BY A COMMA

Written token against token a run of marks cannot be read, so SD puts a comma between them. A text mark, a text mark and a value mark, one after another, is:

```
~!,~!,~~
```

Type the comma yourself when you enter marks in a row. It is a separator and not data — which is why a literal comma standing in exactly that position, between two marks, is written `~,`.

THE CONVERSION IS LOSSLESS

No record is refused and none is mangled, whatever it contains. A tilde is written `~~` **only where the character after it would make the pair look like a token** — another `~`, a backtick, a `!`, a `-`, a `,`, or a mark. Everywhere else a tilde is left exactly as you wrote it, so `a~b` is still `a~b` and ordinary source reads normally.

Two gates, and both are separate from the verb

the VOC tier	decides whether you have the verb at all
---------------------	--

<code>os.users</code> field 2	the <code>OS.EXECUTE</code> field, decides whether it may run
--------------------------------------	---

An editor runs outside SD, so it needs operating-system permission that `ed` does not. It comes from a record in the system file `os.users` whose field 2 reads `yes`, **and only an administrator can put one there**.

An administrator gets one without asking. An account created with the **ADMINISTRATOR** tier — which is the tier of the account SD's installer makes for whoever installs it — is written into `os.users` as it is created, with both fields `yes`. So an administrator reaches the operating system **without elevating**, and these two verbs work in an ordinary session.

For an administrator it is a rule, and it cannot be turned off. For every other tier it is a grant, and there are keywords for it:

<code>create.account user</code> <i>name</i> ... <code>os-on</code>	give the new account <code>OS.EXECUTE</code> — and these two verbs
---	--

<code>create.account user</code> <i>name</i> ... <code>sh-on</code>	give it the <code>sh</code> verb
---	----------------------------------

<code>modify.account</code> <i>name</i> <code>os-on</code> <code>os-off</code>	change it afterwards
--	----------------------

<code>modify.account</code> <i>name</i> <code>sh-on</code> <code>sh-off</code>	the same for the <code>sh</code> verb
--	---------------------------------------

They are four switches over two fields rather than four names for one state, so `sh-off` leaves `OS.EXECUTE` alone.

`modify.account` **needs an elevated session**, as it always has — you elevate to grant somebody the right not to have to.

The four refuse an administrator, in both directions. An administrator has all three routes — `ssh`, the API and the operating system — as a rule, and none of them is `modify.account`'s to change:

```
:modify.account don os-off
don is an administrator and always reaches the operating system
```

It is the same refusal `ssh`, `api`, `both` and `none` already give for an administrator, and it is deliberate rather than an oversight: the whole purpose of the tier is that it grants unlimited access.

The record is ordinary data, so an administrator can also edit it by hand with `ed os.users` *name* from `sdsys`. An account of any other tier starts with no record and is refused until somebody grants it.

An elevated session passes anyway, whatever the file says — otherwise an empty `os.users` would lock the machine's own administrator out.

And a session with no terminal is refused before anything is written — an API session, or a script driving SD down a pipe:

```
:edit bp zzed
edit needs a terminal to draw on, and this session has none.
ed, the line editor, works anywhere.
```

Both usage errors name the verb you typed rather than the program behind it:

```
:edit
No file name specified.  Usage: edit {dict} <file> <record>
:edit bp
No record name specified.  Usage: edit {dict} <file> <record>
```

Who has these verbs

`edit`, `micro` and `ed` are all **programmer verbs**. A standard account has none of them.

See also

SD TCL - The micro Screen Editor · SD TCL - The ed Line Editor · SD TCL - Files and Records.